



TECNOLOGICO DE ESTUDIOS SUPERIORES DE ECATEPEC
DIVISION DE INFORMATICA

Proyecto integrador: Desarrollo de videojuego arcade web

Materia: Desarrollo de Aplicaciones Web

Nombre del Proyecto: *Nitro Island Racing*

Integrantes

- Encarnacion Hernández Ernesto
- Miranda Alatraste Josue Manuel

Grupo: 15501

Objetivo del Videojuego

El objetivo principal de **Turbo Circuit 117** es desarrollar un videojuego de carreras con una perspectiva pseudo-3D utilizando tecnologías web como HTML5 Canvas, JavaScript, CSS3 y Node.js. El proyecto busca crear una experiencia similar a la de los videojuegos de carreras arcade clásicos, pero desarrollada desde cero utilizando herramientas y recursos que permitan ejecutarlo directamente desde un navegador mediante un servidor local.

Uno de los principales objetivos es conseguir que una pista representada en un espacio 2D pueda dar la sensación de profundidad y movimiento. Para lograrlo, se utilizan diferentes cálculos matemáticos que permiten modificar el tamaño y la posición de los segmentos de la carretera conforme se acercan o se alejan del jugador. Además de la parte gráfica, el videojuego cuenta con diferentes sistemas que permiten controlar el funcionamiento de la carrera. Entre ellos se encuentran el sistema de aceleración y frenado, el movimiento lateral del vehículo, la fricción cuando el jugador sale de la pista, las colisiones con otros vehículos y obstáculos, el sistema de vueltas y el cálculo de la posición del jugador dentro de la carrera.

También se implementaron vehículos rivales controlados mediante una inteligencia artificial básica. Estos vehículos tienen su propia velocidad y posición dentro del circuito, permitiendo que la carrera tenga movimiento y que el jugador tenga que competir contra ellos. Finalmente, otro de los objetivos fue integrar todos estos elementos en un solo proyecto funcional, utilizando Node.js para ejecutar el servidor local y permitir que los archivos del videojuego puedan cargarse correctamente desde el navegador.

Descripción General del Funcionamiento

Al iniciar el videojuego, el jugador aparece en la línea de salida junto con 8 vehículos rivales, formando un total de 9 participantes en la carrera. Antes de comenzar el movimiento de los vehículos se muestra una cuenta regresiva de 3, 2, 1, durante la cual los vehículos rivales permanecen detenidos en sus posiciones iniciales. Cuando la cuenta termina y aparece el mensaje "GO", comienza oficialmente la carrera. A partir de ese momento, el jugador puede controlar su vehículo mediante el teclado. Los controles permiten acelerar, frenar y desplazarse hacia los lados para tomar las curvas del circuito y evitar los diferentes obstáculos.

Mientras el jugador avanza, el juego actualiza constantemente diferentes elementos de la carrera. Se calcula la velocidad del vehículo, su posición dentro del circuito, el movimiento de los rivales, las colisiones y el tiempo transcurrido. La pista está formada por diferentes segmentos que permiten representar curvas y cambios de elevación. Mediante el tamaño y la posición de estos segmentos se crea la sensación de profundidad característica de la perspectiva pseudo-3D. A lo largo del circuito también aparecen diferentes elementos del escenario, como palmeras y rocas, que ayudan a representar el ambiente de la pista. Algunos de estos elementos funcionan únicamente como decoración, mientras que las rocas pueden representar obstáculos para el jugador.

Durante la carrera, la interfaz muestra información importante mediante el HUD, como la velocidad actual, el tiempo transcurrido, la posición del jugador y la vuelta en la que se encuentra. El objetivo es completar las 3 vueltas del circuito y llegar a la meta. Al finalizar, el juego muestra el resultado de la carrera y el tiempo total que tardó el jugador en completar el recorrido.

Tecnologías Utilizadas

Para desarrollar **Turbo Circuit 117** se utilizaron diferentes tecnologías relacionadas con el desarrollo web y la programación de videojuegos.

- **HTML5 Canvas:** Se utilizó **HTML5 Canvas** como elemento principal para representar gráficamente el videojuego. El Canvas permite dibujar directamente diferentes elementos en pantalla mediante JavaScript, como la carretera, el cielo, las montañas, los vehículos, las palmeras, las rocas y los diferentes elementos de la interfaz.

También permite actualizar constantemente la imagen mostrada en pantalla para producir el movimiento de la carrera.

- **JavaScript:** JavaScript es el lenguaje principal utilizado para desarrollar la lógica del videojuego. Con este lenguaje se programaron los controles del jugador, la aceleración, el frenado, el movimiento de los vehículos rivales, las colisiones, el sistema de vueltas, el cronómetro y el renderizado de los elementos.

También se utilizaron funciones de JavaScript para realizar diferentes cálculos matemáticos necesarios para generar la perspectiva pseudo-3D de la pista.

- **Node.js:** Node.js se utilizó para crear un servidor local que permite ejecutar el proyecto desde el navegador. De esta manera, los archivos HTML, JavaScript, CSS e imágenes se cargan desde un entorno local organizado.

El servidor también facilita las pruebas del videojuego durante su desarrollo.

- **CSS3:** CSS3 se utilizó principalmente para complementar la presentación visual de la aplicación. Permite establecer estilos, posiciones y características visuales de los elementos que acompañan al Canvas.

Estructura del Proyecto

El proyecto se organizó mediante diferentes carpetas y archivos, con el objetivo de mantener separados los elementos principales del videojuego y facilitar su modificación durante el desarrollo.

La estructura utilizada es la siguiente:

```
-----  
nitro-island-racing/  
|  
|—— index.html  
|—— app.js  
|  
|—— css/  
|   |—— styles.css  
|  
|—— js/  
|   |—— datos/  
|   |   |—— vehiculo.js  
|   |  
|   |—— juego/  
|   |   |—— audio.js  
|   |   |—— jugador.js  
|   |   |—— motor.js  
|   |   |—— obstaculos.js  
|   |   |—— pista.js  
|   |   |—— publico.js  
|   |   |—— rivales.js  
|   |
```

```
|   └── pantallas/
|       ├── carga.js
|       ├── configuracion.js
|       ├── garaje.js
|       ├── menu.js
|       ├── taller.js
|       └── tutorial.js
|
|── assets/
|   ├── controles/
|   ├── pistas/
|   └── sprites/
|       ├── alerones/
|       ├── autos/
|       ├── garaje/
|       ├── llantas/
|       └── publico/
|
|── server/
|   └── server.js
```

--server.js--

Es el archivo encargado de iniciar el servidor local mediante Node.js. Su función principal es permitir que el proyecto pueda ejecutarse desde el navegador y que los diferentes archivos y recursos puedan ser cargados correctamente.

--index.html--

Es la página principal del proyecto. Dentro de este archivo se encuentra la estructura inicial de la aplicación y el elemento ****Canvas**** donde se representa el videojuego.

--juego.js--

Contiene la mayor parte de la lógica del videojuego. En este archivo se encuentran las funciones relacionadas con el movimiento del jugador, los vehículos rivales, las colisiones, el renderizado, las vueltas, el tiempo y los diferentes estados de la carrera.

--Carpeta assets--

Esta carpeta contiene los recursos gráficos utilizados por el videojuego. Dentro de ella se encuentran los sprites correspondientes al vehículo del jugador, los vehículos rivales, las palmeras y las rocas utilizadas como obstáculos dentro del circuito. Esta organización permite mantener los recursos gráficos separados del código y facilita su localización cuando es necesario modificarlos o reemplazarlos.

Arquitectura General

El funcionamiento de **Turbo Circuit 117** se basa en un ciclo principal de juego, conocido como Game Loop, que permite actualizar y dibujar continuamente todos los elementos que aparecen en pantalla. Este ciclo se ejecuta mediante ``requestAnimationFrame``, una función de JavaScript que permite solicitar al navegador que vuelva a ejecutar el proceso de actualización y renderizado. De esta manera, el juego puede mantener un movimiento continuo durante la carrera.

En cada ciclo se realizan principalmente dos procesos:

---Actualización de estados (Update)---

En esta parte se actualiza la información interna del videojuego. Se revisan las teclas presionadas por el jugador y, dependiendo de ellas, se modifica la velocidad y posición del vehículo. También se actualizan la posición de los vehículos rivales, el desplazamiento del circuito, el tiempo de la carrera, las vueltas completadas y los diferentes estados relacionados con las colisiones. Durante esta etapa también se comprueba si el jugador se encuentra fuera de los límites de la pista o si ha tenido contacto con algún vehículo u obstáculo.

----Renderizado (Draw)---

Después de actualizar los datos del juego, se procede a dibujar nuevamente la escena en el Canvas. Primero se representa el fondo, incluyendo el cielo, los degradados y las montañas. Posteriormente se dibujan los diferentes segmentos de la carretera, tomando en cuenta sus curvas y cambios de elevación. Después se colocan los elementos del escenario, como las palmeras y las rocas, y posteriormente los vehículos rivales. Finalmente se dibuja el vehículo del jugador y la interfaz de información o HUD. Este proceso se repite continuamente mientras la carrera se encuentra activa, permitiendo que los cambios de posición y movimiento puedan observarse en tiempo real.

Actividades Realizadas Paso a Paso

El desarrollo de ****Turbo Circuit 117**** se realizó mediante diferentes etapas, comenzando desde el diseño básico del videojuego hasta la integración de todas sus funciones.

- **Fase 1. Investigación y diseño inicial**

Primero se investigó el funcionamiento de los videojuegos de carreras con perspectiva pseudo-3D y la forma en que se puede representar una carretera con profundidad utilizando un Canvas en 2D. También se definió la estructura inicial del proyecto, incluyendo los archivos principales y las carpetas donde se almacenarían los recursos gráficos.

- **Fase 2. Creación del circuito**

Posteriormente se desarrolló el sistema de segmentos de la pista. Cada segmento contiene información que permite determinar características como su longitud, curvatura y elevación. Mediante estos datos se consiguió representar diferentes partes del circuito y crear curvas, pendientes y cambios en la perspectiva de la carretera.

- **Fase 3. Integración de los recursos gráficos**

Después se agregaron los sprites correspondientes al vehículo del jugador, los vehículos rivales, las palmeras y las rocas. Durante esta etapa también se implementó el procesamiento de las imágenes para eliminar los fondos blancos que aparecían en algunos de los sprites y permitir que estos pudieran integrarse correctamente con el resto de la pista.

- **Fase 4. Implementación de los controles y movimiento**

Una vez que la pista y los elementos gráficos funcionaban, se implementaron los controles del jugador. Se agregó la aceleración, el frenado y el movimiento hacia los lados. También se implementó la pérdida de velocidad cuando el vehículo sale de la carretera, haciendo que el jugador tenga que mantenerse dentro de los límites de la pista.

- **Fase 5. Implementación de los vehículos rivales**

Posteriormente se agregaron los vehículos controlados por la computadora. Cada rival cuenta con una posición y velocidad propias. También se agregó una pequeña variación en su comportamiento para evitar que todos los vehículos avanzaran exactamente de la misma manera.

- **Fase 6. Colisiones y sistema de carrera**

Después se implementaron las colisiones con los rivales y con los obstáculos del circuito. También se agregó el sistema de vueltas, el cálculo de posiciones, el cronómetro y los diferentes elementos del HUD necesarios para mostrar información durante la carrera.

- **Fase 7. Servidor local y pruebas**

Finalmente se configuró el servidor local mediante Node.js y se realizaron diferentes pruebas para comprobar que las imágenes, los controles, los rivales, las colisiones y el circuito funcionaran correctamente. Durante esta etapa también se corrigieron diferentes errores encontrados durante las pruebas y se unificó la lógica principal del videojuego.

Diseño de las Pantallas

El videojuego cuenta con diferentes estados visuales que se presentan dependiendo del momento de la carrera.

- **Pantalla de inicio y ejecución**

El videojuego se ejecuta dentro de una página web que contiene el Canvas principal. El área utilizada para representar el juego tiene unas dimensiones de 1000 × 600 píxeles. Dentro del Canvas se muestran el circuito, los vehículos, el escenario y los diferentes elementos de la interfaz.

- **Pantalla de cuenta regresiva**

Antes de comenzar la carrera se muestra una cuenta regresiva con los números 3, 2 y 1. Durante este momento los vehículos permanecen en sus posiciones iniciales para que la carrera no comience antes de tiempo. Al terminar la cuenta regresiva aparece el mensaje “GO”, indicando que el jugador ya puede comenzar a acelerar.

- **Pantalla de carrera**

Es la pantalla principal del videojuego. En ella se representa el circuito y todos los elementos que forman parte de la carrera. Además de la pista y los vehículos, se muestra un HUD con información importante para el jugador, como la velocidad actual, el tiempo transcurrido, la posición dentro de la carrera y la vuelta actual. La posición se muestra, por ejemplo, como 1/9, indicando que el jugador se encuentra en primer lugar de un total de nueve participantes.

- **Pantalla de finalización**

Cuando el jugador completa las vueltas establecidas, el juego muestra la finalización de la carrera y el tiempo total que se registró durante el recorrido. Esta pantalla permite conocer el resultado obtenido después de completar el circuito.

Descripción de las Mecánicas Implementadas

Para que el videojuego pueda funcionar como una carrera, se implementaron diferentes mecánicas relacionadas con el movimiento, el control del vehículo y el avance dentro del circuito.

- **Aceleración**

El jugador puede aumentar progresivamente la velocidad de su vehículo utilizando la tecla W o la flecha hacia arriba. La velocidad no aumenta de manera instantánea, sino que se modifica progresivamente hasta alcanzar el límite establecido para el vehículo.

- **Frenado**

El jugador también puede reducir la velocidad utilizando el control correspondiente de frenado. Esto permite disminuir la velocidad antes de entrar a una curva o cuando sea necesario evitar un obstáculo.

- **Movimiento lateral**

El vehículo puede desplazarse hacia la izquierda o hacia la derecha mediante las teclas A y D o las flechas laterales. Este movimiento permite al jugador posicionarse dentro de la carretera, tomar las curvas y evitar obstáculos y otros vehículos.

- **Fricción fuera de la pista**

Cuando el jugador se desplaza fuera de los límites de la carretera, el vehículo comienza a perder velocidad. Esta mecánica busca evitar que el jugador pueda recorrer el circuito por fuera de la pista sin ninguna consecuencia.

- **Sistema de vueltas**

La carrera está configurada para completar 3 vueltas. El juego utiliza la posición del jugador dentro del circuito para determinar cuándo se ha completado una vuelta. Al recorrer la longitud correspondiente a la pista, el contador de vueltas aumenta hasta llegar a la vuelta final.

- **Posicionamiento**

Durante la carrera se calcula la posición del jugador con respecto a los vehículos rivales. Dependiendo de cuánto haya avanzado cada vehículo dentro del circuito, el jugador puede encontrarse en diferentes posiciones, desde 1/9 hasta 9/9.

Explicación de los Algoritmos Utilizados

Para conseguir el funcionamiento de **Turbo Circuit 117** fue necesario utilizar diferentes algoritmos y cálculos dentro del código. Estos permiten representar la pista, generar la sensación de profundidad y procesar los recursos gráficos utilizados en el videojuego.

- **Algoritmo de segmentación y perspectiva de la pista**

La pista se encuentra dividida en diferentes segmentos que contienen información sobre su longitud, curvatura y elevación. Durante el renderizado, el juego toma como referencia la posición actual del jugador y calcula la distancia de los segmentos que se encuentran delante de él. Dependiendo de esta distancia, los segmentos se dibujan con diferentes tamaños y posiciones. Para conseguir una apariencia de profundidad se utiliza un cálculo basado en la distancia del segmento. Entre estos cálculos se encuentra una función exponencial mediante `Math.pow`, utilizada para modificar la profundidad visual.

De esta manera, los segmentos que se encuentran cerca del jugador aparecen más grandes, mientras que los que se encuentran cerca del horizonte aparecen más pequeños. Esto permite crear la sensación de que la carretera se extiende hacia el fondo aunque todo el escenario se esté dibujando sobre un Canvas 2D.

- **Algoritmo de procesamiento de imágenes**

También se implementó un algoritmo para eliminar automáticamente los fondos blancos de algunos sprites. El programa analiza los píxeles de la imagen y revisa sus valores RGB. Cuando los tres valores se encuentran por encima de un determinado límite, el píxel se considera suficientemente cercano al color blanco y su canal alfa se establece en `0`. Esto hace que dicho píxel se vuelva transparente y permite que el sprite pueda colocarse sobre la carretera sin mostrar un cuadro blanco alrededor de la imagen.

El funcionamiento principal del algoritmo es el siguiente:

javascript

```
for (let i = 0; i < pixels.length; i += 4) {  
  if (pixels[i] > 245 && pixels[i + 1] > 245 && pixels[i + 2] > 245) {  
    pixels[i + 3] = 0;  
  }  
}
```

En este proceso, los valores de la imagen se revisan de cuatro en cuatro porque cada píxel contiene información correspondiente a los canales rojo, verde, azul y alfa (RGBA). Al modificar el canal alfa a `0`, el píxel deja de ser visible. Estos algoritmos son importantes para conseguir tanto la perspectiva del circuito como la correcta integración de los elementos gráficos dentro del juego.

Descripción de la Lógica de los Enemigos

Los vehículos rivales son controlados automáticamente por el propio videojuego. Su comportamiento se encuentra separado del control del jugador y cada vehículo cuenta con información propia sobre su posición, desplazamiento lateral y velocidad. Los rivales se almacenan dentro de un arreglo, donde cada vehículo cuenta principalmente con valores como:

* **`position`**: indica su avance dentro del circuito.

* **`x`**: representa su posición lateral en la pista.

* **`speed`**: determina la velocidad a la que avanza.

Durante la cuenta regresiva, los vehículos rivales permanecen detenidos. Esto se controla mediante el estado de la carrera, evitando que puedan avanzar antes de que aparezca el mensaje de inicio. Una vez que comienza la carrera, sus posiciones empiezan a actualizarse de acuerdo con su velocidad.

Además, se utilizan pequeñas variaciones pseudoaleatorias mediante `Math.random()` para modificar ligeramente su comportamiento. Esto permite que los rivales no mantengan exactamente el mismo ritmo durante toda la carrera y genera pequeñas diferencias entre sus posiciones. El objetivo de esta lógica no es crear una inteligencia artificial compleja, sino proporcionar un comportamiento suficiente para que los vehículos funcionen como oponentes durante la carrera y puedan competir por diferentes posiciones.

Sistema de Colisiones

El sistema de colisiones permite detectar cuando el vehículo del jugador entra en contacto con otros elementos del circuito. Para esto se comparan las posiciones de los objetos tanto en el recorrido de la pista como en el eje lateral.

- **Colisiones con vehículos rivales**

Para detectar una posible colisión con un rival se compara la diferencia entre la posición del jugador y la posición del vehículo contrario. También se revisa la separación horizontal entre ambos vehículos. Cuando los dos se encuentran suficientemente cerca tanto en el recorrido como lateralmente, se considera que ocurrió una colisión.

Cuando esto sucede, el vehículo del jugador recibe una penalización que puede provocar una reducción de velocidad y una modificación temporal de su avance. Esto hace que los vehículos rivales no sean solamente elementos decorativos, sino que también puedan afectar el desarrollo de la carrera.

- **Colisiones con obstáculos**

Las rocas que aparecen en diferentes zonas de la pista funcionan como obstáculos. Cuando el vehículo del jugador se acerca demasiado a una roca, el juego comprueba la distancia entre ambos elementos. Si se cumplen las condiciones establecidas para la colisión, se activa un efecto de giro del vehículo. Durante este efecto, el vehículo pierde velocidad y se modifica temporalmente su orientación mediante cálculos trigonométricos. Después de cierto tiempo, el vehículo recupera su comportamiento normal. Este sistema permite representar de manera visual las consecuencias de chocar contra un obstáculo.

Sistema de Puntuación y Posicionamiento

El videojuego utiliza un sistema de posicionamiento que permite determinar el lugar que ocupa el jugador durante la carrera. Para calcularlo, se compara la distancia recorrida por el jugador con la distancia recorrida por cada uno de los vehículos rivales. Si un rival tiene una posición de avance mayor que la del jugador, significa que ese vehículo se encuentra por delante.

El juego cuenta cuántos rivales se encuentran adelante y utiliza esta cantidad para determinar la posición actual del jugador. Por ejemplo, si ningún rival ha avanzado más que el jugador, este se encuentra en 1/9. Si existen tres vehículos por delante, el jugador ocuparía la posición 4/9.

Este cálculo se actualiza durante la carrera, por lo que la posición puede cambiar constantemente dependiendo del avance de cada participante. Además del posicionamiento, el juego registra el tiempo empleado para completar la carrera, permitiendo mostrar el resultado final al terminar el circuito.

Uso de Node.js

Node.js se utilizó para crear el servidor local necesario para ejecutar el proyecto durante su desarrollo y sus pruebas. El archivo principal encargado de esta función es `server.js`, el cual permite levantar un servidor local desde el equipo donde se encuentra almacenado el proyecto.

Una vez iniciado el servidor, el videojuego puede abrirse desde el navegador utilizando una dirección local. De esta manera, el archivo `index.html` y los demás recursos del proyecto se cargan desde el servidor. El uso de Node.js también permite mantener organizada la estructura del proyecto y facilita la carga de los archivos JavaScript, hojas de estilos e imágenes utilizadas por el videojuego.

Durante el desarrollo, el servidor local fue utilizado para comprobar que las diferentes rutas de los recursos fueran correctas y que los sprites pudieran cargarse correctamente. Por lo tanto, Node.js funciona principalmente como el entorno que permite servir y ejecutar el proyecto web de manera local mientras se desarrolla y prueba el videojuego.

Almacenamiento de Información

El videojuego no utiliza una base de datos externa para almacenar la información de las partidas. Los datos necesarios para ejecutar una carrera se mantienen temporalmente en la memoria durante la sesión activa del juego.

Entre los datos que se mantienen durante la ejecución se encuentran:

- Velocidad actual del jugador.
- Posición del jugador dentro del circuito.
- Posiciones y velocidades de los vehículos rivales.
- Vuelta actual.
- Tiempo transcurrido.
- Estado de la carrera.
- Estado de las colisiones.
- Información relacionada con los sprites procesados.

Estos datos son utilizados constantemente por las funciones de actualización y renderizado para determinar qué debe ocurrir y qué debe mostrarse en pantalla. Al finalizar la ejecución del videojuego, esta información temporal deja de estar disponible, ya que el proyecto no cuenta actualmente con un sistema de almacenamiento permanente de resultados o partidas.

También se utiliza una caché para algunos de los sprites que han sido procesados, evitando realizar nuevamente el mismo procesamiento de imágenes cuando ya se dispone de una versión preparada para utilizarse durante el juego.

Pruebas Realizadas

Durante el desarrollo de **Turbo Circuit 117** se realizaron diferentes pruebas para comprobar que las funciones principales del videojuego trabajaran correctamente y que los diferentes elementos estuvieran sincronizados.

- **Prueba de carga de sprites**

Se verificó que los archivos de imagen utilizados por el videojuego fueran encontrados correctamente por el servidor local. Esta prueba permitió detectar posibles errores en las rutas de los archivos y comprobar que los vehículos, palmeras y rocas pudieran cargarse dentro del Canvas.

- **Prueba de la cuenta regresiva**

Se comprobó que la carrera no comenzara antes de tiempo. Durante los números 3, 2 y 1, los vehículos permanecen detenidos en la línea de salida. Después de finalizar la cuenta regresiva y aparecer el mensaje “GO”, se permite el movimiento de los vehículos.

- **Prueba del movimiento del jugador**

Se probaron los controles del vehículo para comprobar que la aceleración, el frenado y el movimiento lateral respondieran correctamente a las teclas configuradas. También se comprobó el comportamiento del vehículo al salir de los límites de la carretera y la pérdida de velocidad producida por la fricción.

- **Prueba de las curvas**

Se realizaron recorridos sobre diferentes partes del circuito para comprobar que la carretera pudiera representar correctamente sus curvas y cambios de dirección. También se verificó que los elementos del escenario siguieran el desplazamiento generado por la curvatura de la pista.

- **Prueba de los vehículos rivales**

Se comprobó que los rivales permanecieran detenidos durante la cuenta regresiva y comenzaran a desplazarse cuando la carrera iniciara. También se revisó que sus posiciones fueran actualizadas correctamente y que pudieran cambiar de posición con respecto al jugador.

- **Prueba de colisiones**

Finalmente, se realizaron pruebas de contacto con otros vehículos y con los obstáculos del circuito. Se verificó que las colisiones produjeran los efectos establecidos, como la reducción de velocidad y el giro del vehículo cuando se golpea una roca.

Problemas Encontrados

Durante el desarrollo del proyecto se presentaron diferentes problemas que tuvieron que ser identificados y corregidos para conseguir que el videojuego funcionara correctamente.

- **Fondos blancos en los sprites**

Uno de los problemas encontrados ocurrió con algunas de las imágenes utilizadas para los vehículos y otros elementos del escenario. Los sprites tenían fondos blancos sólidos que, al ser dibujados sobre el Canvas, aparecían como cuadros blancos alrededor de los objetos y cubrían parte de la pista.

- **Movimiento prematuro de los rivales**

Otro problema ocurrió durante la cuenta regresiva de la carrera. En una de las primeras versiones, los vehículos rivales comenzaban a avanzar antes de que terminara la cuenta regresiva. Esto provocaba que los vehículos ya hubieran cambiado de posición cuando aparecía el mensaje de inicio de la carrera.

- **Problemas con la organización del código**

Durante el desarrollo también se presentaron problemas relacionados con la separación de diferentes partes del código en varios archivos JavaScript. Algunas funciones dependían de variables o elementos definidos en otros archivos, lo que provocaba errores de referencia y dificultades para mantener correctamente conectado el funcionamiento del juego.

Estos problemas hicieron necesario revisar la organización del código y la forma en que se comunicaban las diferentes funciones.

Soluciones Implementadas

Después de identificar los problemas encontrados durante las pruebas, se implementaron diferentes soluciones para mejorar el funcionamiento del videojuego.

- **Eliminación de fondos blancos**

Para solucionar el problema de los fondos blancos de los sprites se desarrolló la función ``getSpriteWithoutWhite``. Esta función analiza los píxeles de las imágenes y detecta aquellos que tienen valores cercanos al blanco. Cuando encuentra uno de estos píxeles, modifica su canal alfa para hacerlo transparente.

De esta manera, solamente permanece visible el objeto principal del sprite y puede colocarse sobre la pista sin mostrar el fondo blanco.

- **Corrección del movimiento de los rivales**

Para evitar que los vehículos rivales se movieran durante la cuenta regresiva, se agregó una condición que verifica si la carrera ya comenzó. Mientras la variable correspondiente al estado de la carrera indique que todavía no ha iniciado, la función encargada de actualizar los rivales no modifica sus posiciones. Una vez que la carrera comienza, la actualización vuelve a ejecutarse normalmente.

- **Organización del código**

Para solucionar los problemas de referencias entre diferentes partes del proyecto se decidió unificar la lógica principal del videojuego en un archivo centralizado. Esto permitió tener en un mismo lugar las funciones relacionadas con el movimiento, renderizado, rivales, colisiones y demás sistemas principales. Con esta organización fue más sencillo localizar errores y realizar modificaciones sin afectar otras partes del proyecto.

Resultados Obtenidos

Como resultado del desarrollo se obtuvo un videojuego de carreras web funcional llamado **Turbo Circuit 117**. El proyecto permite iniciar una carrera con nueve vehículos, incluyendo al jugador, y cuenta con una pista capaz de representar curvas, cambios de elevación y diferentes elementos del entorno mediante una perspectiva pseudo-3D.

También se implementaron correctamente los principales sistemas necesarios para desarrollar una carrera, como el movimiento del jugador, aceleración, frenado, desplazamiento lateral, fricción fuera de la pista, vehículos rivales, colisiones, vueltas y posicionamiento. La interfaz permite visualizar información importante durante la carrera, incluyendo la velocidad, el tiempo, la posición y la vuelta actual.

Las pruebas realizadas permitieron corregir problemas relacionados con los sprites, el inicio de los rivales y la organización del código. Después de realizar estas correcciones, los diferentes sistemas pudieron funcionar de manera conjunta dentro del servidor local. El resultado final es un videojuego que combina programación, diseño gráfico y diferentes cálculos matemáticos para crear una experiencia de carreras con estilo arcade utilizando tecnologías web.

Conclusiones

El desarrollo de **Turbo Circuit 117** permitió poner en práctica diferentes conocimientos de programación y desarrollo web, principalmente mediante el uso de JavaScript y HTML5 Canvas. Uno de los principales retos del proyecto fue conseguir que una pista dibujada en un espacio 2D pudiera generar una sensación de profundidad y movimiento. Para lograrlo fue necesario trabajar con segmentos, posiciones, curvas y diferentes cálculos para modificar la perspectiva de la carretera.

También fue importante implementar los diferentes sistemas que forman parte de una carrera, como el movimiento del vehículo, los rivales, las colisiones, las vueltas y el posicionamiento. Cada uno de estos elementos tuvo que funcionar de manera coordinada para conseguir que el videojuego pudiera desarrollarse correctamente. Durante el proceso también se presentaron algunos problemas, especialmente con los fondos de los sprites, el movimiento de los rivales durante la cuenta regresiva y la organización del código. Resolver estos problemas permitió comprender mejor la importancia de realizar pruebas constantes durante el desarrollo de un proyecto.

El uso de Node.js permitió ejecutar el videojuego mediante un servidor local y facilitó las pruebas del proyecto durante su desarrollo. En conclusión, **Turbo Circuit 117** permitió aplicar de manera práctica diferentes conocimientos relacionados con la programación, el manejo de gráficos en Canvas, la lógica de videojuegos y el desarrollo web. Además, el proyecto permitió trabajar en equipo y aprender de los errores que fueron apareciendo durante el proceso hasta conseguir una versión funcional del videojuego.

Sprites

Autos:



Llantas:



Alerones:



Obstáculos:

